Pressure-testing the paper: what to build, in order

Ordered by your priority. Each item has **Search** (brute force that works today) and **Solve** (construct it directly). Sections are independent — build top-down.

---

0. The prerequisite: constructing FTR/DAM pairs

An `(f, g)` pair is fully determined by two things:

design variable	what it controls
contingency sets `C_f`, `C_g`	**coverage** differences
limit vectors `b_f`, `b_g` on shared rows	**level** differences

and the disagreement kinds follow mechanically:

- `c ∈ C_g \ C_f` → coverage difference feeding **U** (FTR blind to a case DAM enforces)
- `c ∈ C_f \ C_g` → coverage difference feeding **V**
- shared `c`, `b_f < b_g` → level difference feeding **V**; `b_f > b_g` → level feeding **U**

So *every* target below is a statement about which rows disagree and which bind. That is the whole design space. Nothing else matters.

Search. Enumerate `C_f, C_g ⊆ candidates` (after the redundancy screen below), sweep a derate `α` on shared rows, score with `gap_summary`. With ~5 surviving contingencies that's `4^5 × |α|` cells — trivial.

Solve. Pick the disagreement pattern you want, then use `solve_limit_design` (generalized, §2) to find limits realizing it.

Screen first — this is exact and needs no sweep. Row `i` is redundant iff

```
max { kᵢᵀq : Kq ≤ b, row i dropped } ≤ bᵢ
```

One LP per row. A redundant contingency changes `Q(b)` not at all, so it cannot produce *any* of the structures below. Drop those candidates before sweeping. Cheap alternative: the union of tight sets over `faces(model)` is exactly the non-redundant rows — free if you already enumerated faces.

---

1. Both failure modes live (`U > 0` and `V > 0`)

Objective: maximize `min(U, V)`.

Search. Grid over `(C_f, C_g, α)` from §0, take `min(U,V)` from `gap_summary`. This is how you got `mixed` at α=0.85 on the toy; the same sweep works here.

Solve. `U > 0` needs a row where `g` is strictly tighter *and binds at the intersection optimum*; `V > 0` needs the mirror. So: choose one binding pattern containing a row only `g` enforces, and one containing a row only `f` enforces, and solve for limits making **both** patterns exact in one program — `solve_limit_design` already takes multiple named patterns sharing one `b`. That is the feature you need, already written.

Watch: binding is the whole game. At α=0.75 the toy's extra contingency never bit and `U = 0` in every scenario. Non-binding ⇒ no witness.

---

2. Attribution above constraint level, **spanning contingencies**

You are right that I was wrong: with contingencies, rows from *different* cases can share a block. The cycle space of `A` governs the base case only; each contingency has its own `H_c`, so the object is the stacked system.

The block condition is a statement about the trade matrix `C`, whose columns are $c_i = [\bar{k}_i ; b_i]$ — the mean-removed PTDF row stacked on the limit. Rows `S` share a block iff some `z` supported on `S` has $C z = 0$, i.e. **both**

```
(a)  Σ_{i∈S} z_i k̄_i = 0      ← PTDF only. b-free. Pure linear algebra.
(b)  Σ_{i∈S} z_i b_i = 0      ← LINEAR IN b.
```

That split is the whole answer to "can I solve rather than search":

Search. Enumerate small row subsets `S` spanning ≥2 contingencies, form `C_S`, keep those with `rank(C_S) < |S|`. Cost is a tiny SVD per subset; restrict to $|S| \le 4$ and you are done in seconds. Report whether any surviving `S` touches more than one contingency key — that is the question you want answered.

Solve. Two clean steps, no search:

1. Compute `null({k̄_i}_{i∈S})` from the stacked PTDF alone — condition (a), independent of limits. If empty, `S` can *never* be a block; reject before any optimization. This replaces my (wrong) cycle-space claim with the correct stacked version, and it is still free.
2. For each surviving null vector `z`, add $\Sigma\ z_i\ b_i = 0$ to `solve_limit_design` as **one more linear constraint**. Condition (b) is linear in `b`, so forcing a designated block costs nothing structurally.

De-hardcoding `solve_limit_design`

It is hard-coded in four ways. Fix them in this order:

1. **Patterns index elements (`for e in range(ell)`), so they cannot name a contingency row.** Change the pattern to carry **global row indices** into the stacked system; sign is then implied by which half of the stack the row is in. This is the change that unlocks everything in this section.
2. **`b` is one vector over elements.** Make it a vector over *rows* (contingency × element), with an optional tie `b[c,e] == b[base,e]` when you want shared limits. Real FTR/DAM pairs differ by post-contingency rating anyway.

3. `b[WD1] == 100`, `b[WD2] == b[WD1]` **are baked in.** Take an `extra` callable `(b, q) -> list[constraint]` so case-specific ties live at the call site.
4. `q[W] ≥ 0, q[N] ≥ 0, q[H] ≤ 0`, **box ±1000 are baked in.** Same treatment — they are an economic story about *this* network, not part of the method.

After (1)–(4) the function is network-agnostic and takes a `NetworkModel`, which already owns the stacked `K` and the contingency list.

---

3. Floor is partial (`0 < floor_ratio < 1`)

You said you have no idea how to test this. It is the easiest one on the list — the floor is binary today for a structural reason you can invert.

`floor_U = Σ μᵢ (bᵢ − (f∧g)ᵢ)` is supported only where the models disagree **and** the row is priced. An unmonitored row forces `μᵢ = 0`, so a coverage difference contributes exactly 0. Hence:

- pure level difference → floor = whole mode → ratio **1.0**
- pure coverage difference → floor = 0 → ratio **0.0**

Today `mixed` has level→V and coverage→U — *different* modes, so each ratio is still 0 or 1.

Solve. Put a level difference **and** a coverage difference into the **same** mode. Concretely: `g` enforces contingency `c` that `f` does not (coverage→U), **and** `f` is looser than `g` on some shared, binding row (level→U). Then `0 < floor_U < U` by construction. No search at all.

Search. If you would rather confirm than construct: scan pairs for `len(differences(f,g)["level_U"]) > 0 and len(...["coverage_U"]) > 0`, then read `floor_ratio_U` from `gap_summary`.

---

4. Repairs sub/superadditive

`U^(SuT)` vs `U^(S) + U^(T)` for disjoint `S, T` (`prop:repair_nonadditive`).

Search. This one is genuinely cheap and I would just brute force it. Take `D =` the disagreeing rows, enumerate disjoint pairs of singletons (or of small sets), and compute all three repair values. `repair_value(..., base=h_model)` reuses the unrepaired solve, so each is **one** LP. `|D|²/2` LPs — seconds. Report `U^(SuT) − U^(S) − U^(T)`; sign gives you sub vs super, and you want witnesses of *both* signs.

Solve. Partial, and worth knowing: repairs interact only through rows that bind *together*. If `S` and `T` price into different blocks and no injection trades between them, the repair is additive. So look for non-additivity where `S` and `T` land in the **same block** (superadditive candidates: relaxing both opens a direction neither opens alone) versus **different blocks joined by a shared binding row** (subadditive candidates: each repair alone already captures the shared row's slack). Use §2's block computation to pick candidates instead of enumerating blindly.

---

5. Primal invariance has content (`identified = False`)

Lower priority per you, but note it is currently **unwitnessed anywhere** — every block reports `True`, because every intersection optimum has been a vertex, where `prop:primal_invariance` holds *vacuously*.

Solve (no search needed). The condition has content only when `Q(f∧g)`'s optimal face has positive dimension. Force that: take `d` = a single row's normal of the meet. Then `d` is parallel to that facet and the optimal face is the whole facet, not a vertex. `faces()` already hands you facet normals.

Search. Sweep `d` over the exposing directions from `faces(meet)` plus the facet normals, and record where `identified` goes False.

Visualization, now that plots are gone

texas5 is 4-D; `viz` is 3-node only and a projection would be dishonest (its boundary edges are not images of constraints).

1. **`faces(model)` is the picture as a table** — every vertex with its tight set. 32 rows for texas5 base. This is the complete regime map.
2. **Facet census** — for each row: never tight (redundant), sometimes, always. Three lines over `faces()`, and it doubles as the §0 redundancy screen.
3. **A 2-D slice, if you want an actual plot.** A slice is exact where a projection is not. `plane_system` needs one small change: an offset q_0, so $q = q_0 + Tu$ and c becomes $b - Kq_0$. ~4 lines, and then `polygon`, `draw_region` and `draw_constraints` all work unchanged at 5 nodes.

Build order

1. **Redundancy screen** (§0) — exact, no sweep, shrinks everything downstream.
2. **De-hardcode `solve_limit_design`** (§2, steps 1–4) — unlocks §1, §2, §3.
3. **Cross-contingency block predictor** (§2 solve, step 1) — the b-free null space test. Answers your actual question: *are* cross-contingency blocks possible on this network?
4. **Same-mode level+coverage pair** (§3) — one constructed case, kills the binary-floor gap.
5. **Repair non-additivity sweep** (§4) — brute force, cheap.
6. **Slice plotting** (§viz 3) and **primal invariance** (§5) last.

Items 1–3 are the ones that change what you can ask. 4–5 are results you can harvest immediately once 2 lands.